INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT IV

Pointers & User-Defined Data Types

Pointers • Address & dereference operators • Pointer arithmetic • Structures • Unions

IV

▶ What you will learn in this unit

- What is a pointer?
- Address (&) and dereference (*) operators
- Pointer declaration & NULL pointer
- Pointer to pointer
- Pointer & address arithmetic
- Array manipulation using pointers
- Structures: declare, access, nest
- Array of structures
- Pointer to structure (->) & typedef
- Unions & shared memory
- Structure vs union
- Common mistakes & questions

4.1 Pointers: Address and Dereference Operators

◆ Definition

A **pointer** is a variable that stores the **memory address** of another variable. Every variable lives at some address in RAM; a pointer lets you reach that variable **indirectly**.

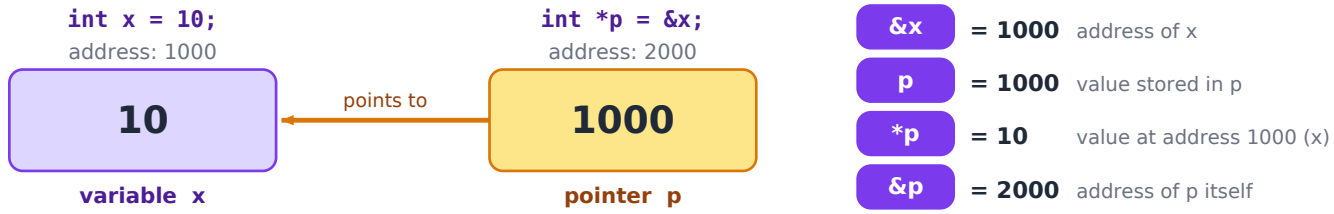


Fig 4.1 — A pointer holds the address of a variable

Operator	Name	Meaning	Example
`&`	Address-of	Gives the address of a variable	`p = &x;`
`*` (in declaration)	Pointer declaration	Creates a pointer variable	`int *p;`
`*` (in expression)	Dereference / indirection	Reads or writes the value at the address	`*p = 25; y = *p;`
`->`	Arrow	Member access through a struct pointer	`ps->marks`

Pointer basics

```c
int x = 10;
int *p = &x;                          // p stores the address of x
printf("x = %d, *p = %d\n", x, *p);   // both give 10
*p = 25;                              // change x via pointer
printf("x = %d\n", x);
printf("Address = %p\n", (void *)&x);  // %p = address
```

OUTPUT

```
x = 10, *p = 10
x = 25
Address = 0x7ffd...
```

- **Syntax:** `datatype *name;` — the data type says what kind of value lives at the address (`int *` → int, `char *` → char).
- **Size of a pointer** is the same for every type: 8 bytes on a 64-bit system (4 bytes on 32-bit).
- **NULL pointer:** `int *p = NULL;` means "points to nothing". Always initialize pointers; test `if (p != NULL)` before use.
- **void pointer:** `void *v;` can hold the address of any type, but must be cast before dereferencing.

⚠ **Common Mistake**
Wild pointer: `int *p; *p = 5;` — p holds garbage, so the write goes to a random place (crash!).
Dangling pointer: pointing to memory that no longer exists (e.g. a local variable after its function returns).

Statement	x	p	*p	Explanation
`int x = 10;`	10	—	—	x created at address 1000
`int *p = &x;`	10	1000	10	p stores x's address
`*p = *p + 5;`	15	1000	15	value at 1000 changed → x changes
`x = 40;`	40	1000	40	x changed → *p sees 40

Why use pointers?	Benefit
Call by reference	A function can modify the caller's variables (Unit V)
Efficiency	Pass the address of big arrays / structures instead of copying
Data structures	Linked lists, trees, graphs are built from pointers



*Fig 4.2 — Pointer to pointer: int x = 10; int *p = &x; int **pp = &p;*

Pointer to pointer

```c
int x = 10, *p = &x, **pp = &p;
**pp = 99;                          // x changes through two levels
printf("%d %d %d", x, *p, **pp);
```

OUTPUT

```
99 99 99
```

4.2 Pointer and Address Arithmetic

Adding 1 to a pointer does **not** add 1 byte — it moves to the **next element**, i.e. adds `sizeof(type)` bytes:

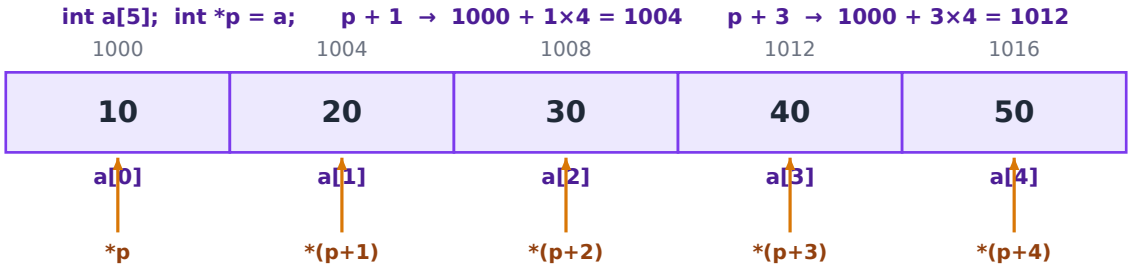


Fig 4.3 — Pointer arithmetic on an int array (each int = 4 bytes)

Operation	Allowed?	Meaning / Example
`p++`, `p--`, `++p`	Yes	Move to next / previous element
`p + n`, `p - n`	Yes	Address n elements ahead / behind: `*(p+2)` = third element
`p2 - p1`	Yes	Number of **elements** between two pointers (same array)
`p1 < p2`, `p1 == p2`	Yes	Compare positions
`p1 + p2`, `p * 2`, `p / 2`	**No**	Meaningless for addresses — compile error

★ Exam Focus
Scaling rule: new address = old address + n × sizeof(pointed type). `char *` moves 1 byte, `short *` 2, `int *` 4, `double *` 8. Precedence: `*p++` = value at p, then move p; `(*p)++` = increase the **value**; `*++p` = move first, then read.

Pointer expressions: *p++, (*p)++, *++p

```c
int a[] = {10, 20, 30}, *p = a;
printf("%d ", *p++);      // prints 10, THEN p moves to a[1]
(*p)++;                   // value at p increases: a[1] becomes 21
printf("%d ", *p);        // 21
printf("%d", *++p);       // p moves to a[2] FIRST, then prints 30
```

OUTPUT

```
10 21 30
```

4.3 Array Manipulation Using Pointers

The name of an array is the address of its first element, so these are all **equivalent**:

Element	Array notation	Pointer notation (p = a)	Address
1st	`a[0]`	`*a` or `*p`	`a` or `&a[0]`
i-th	`a[i]`	`*(a + i)` or `*(p + i)` or `p[i]`	`a + i` or `&a[i]`

Accessing with pointers

```c
int a[] = {10, 20, 30, 40, 50};
int *p = a;                              // no & needed: a already is an address
printf("%d %d %d\n", *p, *(p + 1), *(p + 4));
p++;                                     // p now points to a[1]
printf("%d\n", *p);
printf("%ld", (long)(&a[4] - &a[1]));    // pointer difference = elements
```

OUTPUT

```
10 20 50
20
3
```

Sum and maximum using pointers

```c
int a[5] = {3, 8, 1, 9, 4}, *p = a, *max = a, i, sum = 0;
for (i = 0; i < 5; i++) {
    sum += *(p + i);                     // same as a[i]
    if (*(p + i) > *max) max = p + i;    // remember ADDRESS of biggest
}
printf("Sum=%d Max=%d", sum, *max);
```

OUTPUT

```
Sum=25 Max=9
```

Reverse an array using two pointers

```c
int *l = a, *r = a + 4, t;               // l = first element, r = last element
while (l < r) {                          // pointer comparison
    t = *l;  *l = *r;  *r = t;           // swap the VALUES
    l++;  r--;
}
for (p = a; p < a + 5; p++) printf("%d ", *p);   // traverse
```

OUTPUT

```
4 9 1 8 3
```

String length using a pointer

```c
char *s = "Hello";  int len = 0;
while (*s != '\0') {                     // stop at the null character
    len++;
    s++;
}
printf("Length = %d", len);
```

OUTPUT

```
Length = 5
```

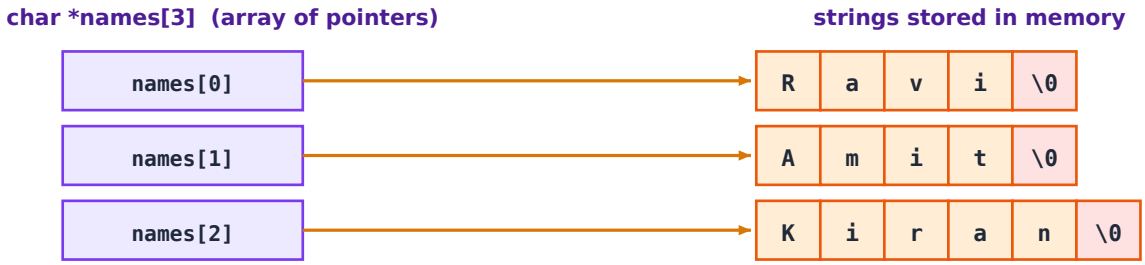


Fig 4.4 — Array of pointers: each element points to a string

String copy with pointers

```c
char src[] = "Hello", dst[10], *s = src, *t = dst;
while (*s != '\0') *t++ = *s++;     // copy char by char, move both pointers
*t = '\0';                          // terminate the new string
printf("%s", dst);
char *names[3] = {"Ravi", "Amit", "Kiran"};
printf(" %s", names[2]);
```

OUTPUT

```
Hello Kiran
```

	Array name `a`	Pointer `p`
Nature	Constant address of 1st element	Variable that stores an address
Can change?	`a++` or `a = p` → **error**	`p++`, `p = a` allowed
`sizeof`	whole array: `sizeof(a)` = 20 bytes (5 ints)	just the pointer: 8 bytes
Memory	Allocated automatically for all elements	Only 8 bytes; must point somewhere valid

● Real-life Use

Pointers allow **call by reference** (functions that change caller variables), fast array / string processing, and are the foundation of linked lists and trees.

4.4 User-Defined Data Types: Structures

◆ Definition

A **structure** (`struct`) groups **different types** of variables under one name. An array holds many values of the **same** type; a structure holds related values of **different** types (e.g. id, name, marks of a student).

Declaring and using a structure

```c
struct Student {            // 1. declare the TEMPLATE (no memory yet)
    int   id;
    char  name[20];
    float marks;
};                          // <-- semicolon is compulsory

struct Student s1 = {101, "Asha", 88.5};   // 2. create variable
s1.marks = 90.0;                           // 3. dot operator
printf("%d %s %.1f", s1.id, s1.name, s1.marks);
scanf("%d", &s1.id);                       // reading a member needs &
```

OUTPUT

```
101 Asha 90.0
```

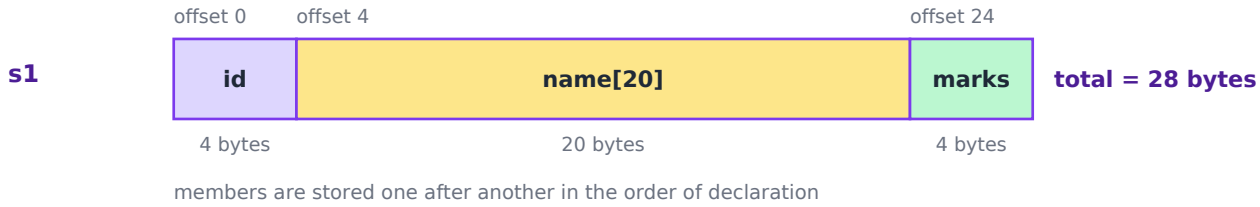


Fig 4.5 — Memory layout of struct Student s1

Style	Code	Note
Tagged	`struct Pt { int x, y; }; struct Pt a;`	Reusable template
With variables	`struct Pt { int x, y; } a, b;`	Declare template + variables
typedef	`typedef struct { int x, y; } Pt; Pt a;`	Short name, no struct keyword

Array	Structure
Same data type elements	Different data types allowed
Access by index `a[2]`	Access by member name `s.marks`
Elements are anonymous	Each member has its own name

Array of structures, nested structures, pointer to structure

Array of structures & arrow operator

```c
struct Student s[3] = {{1,"Ravi",72.5}, {2,"Amit",91.0},
                       {3,"Kiran",65.0}};
int i, top = 0;
for (i = 1; i < 3; i++)
    if (s[i].marks > s[top].marks) top = i;   // s[i].member
printf("Topper: %s (%.1f)\n", s[top].name, s[top].marks);

struct Student *ps = &s[0];       // POINTER to structure
printf("%s %.1f", ps->name, ps->marks);  // (*ps).x
```

OUTPUT

```
Topper: Amit (91.0)
Ravi 72.5
```

Nested structure and typedef

```c
struct Date { int d, m, y; };
struct Employee {
    char name[20];
    struct Date dob;                 // NESTED structure
};
struct Employee e = {"Meena", {14, 8, 2001}};
printf("%s born in %d", e.name, e.dob.y);        // chain of dots

typedef struct { int x, y; } Point;  // shorter name
Point p1 = {3, 4};                   // no 'struct' needed
```

OUTPUT

```
Meena born in 2001
```

- **Dot (.)** is used with a structure **variable**; **arrow (→)** with a structure **pointer**.
- Assigning `s2 = s1;` copies **all** members (even arrays inside). Structures cannot be compared with `==`.
- **Padding:** compilers align members, so `struct {char c; int i;}` occupies **8 bytes**, not 5. Always use `sizeof`.



Fig 4.6 — A pointer to a structure: members are reached with the arrow operator ->

Linear search using a pointer

```c
int a[] = {12, 45, 7, 30, 21}, *p, key = 30, found = 0;
for (p = a; p < a + 5; p++)        // p itself moves
    if (*p == key) { found = 1; break; }
if (found) printf("Found at position %ld", (long)(p - a) + 1);
else       printf("Not found");
```

OUTPUT

```
Found at position 4
```

4.5 User-Defined Data Types: Unions

◆ Definition

A **union** also groups members of different types, but **all members share the same memory**. Its size is that of its **largest member**, and only **one member** holds a valid value at a time.

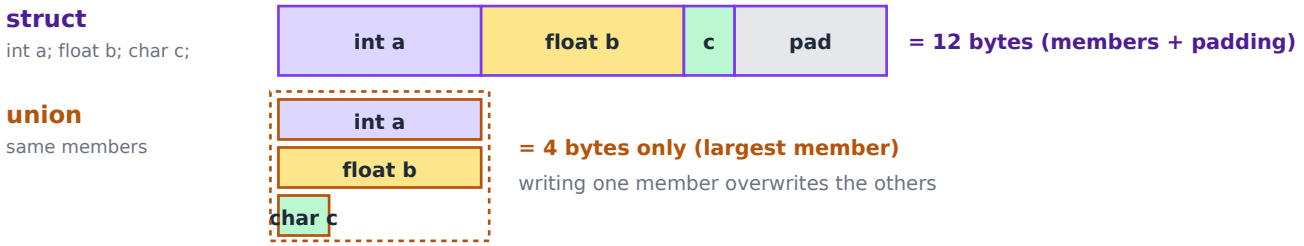


Fig 4.7 — Memory of struct vs union with the same members

Using a union

```c
union Data { int i; float f; char c; };     // declared like a struct
union Data u;
u.i = 65;
printf("%d %c\n", u.i, u.c);                // 65 A  (same byte read as char)
u.f = 3.5;                                   // overwrites u.i
printf("%.1f\n", u.f);
printf("size = %zu", sizeof(u));             // largest member = 4
```

OUTPUT

```
65 A
3.5
size = 4
```

Feature	Structure	Union
Keyword	`struct`	`union`
Memory	Each member has its **own** space	All members **share** one space
Size	Sum of all members (+ padding)	Size of the **largest** member
Active members	All members valid together	Only the **last written** member is valid
Use when	Records: student, employee, point	Saving memory; variant data (packet, register)

● Real-life Use

Structures: student / employee records, linked-list nodes, coordinates, bank accounts. **Unions:** embedded hardware registers, network packet formats, interpreters that store "an int *or* a float *or* a char".

Tagged union (struct + union together)

```c
struct Shape {
    int type;               // TAG: 1 = int, 2 = float
    union { int i; float f; } val;
};
struct Shape v = {2, .val.f = 3.14f};
if (v.type == 1) printf("%d", v.val.i);
else             printf("%.2f", v.val.f);
```

OUTPUT

```
3.14
```

Predict the output (practice)

#	Code	Output
1	`int x=5, *p=&x; *p = *p * 2; printf("%d", x);`	10
2	`int a[]={1,2,3}, *p=a+1; printf("%d", *p + *(p+1));`	5 (2 + 3)
3	`int a[]={4,5,6}; printf("%d", *(a+2) - *a);`	2 (6 − 4)
4	`struct P {int x,y;} p={3,4}; printf("%d", p.x*p.y);`	12
5	`union U {int i; char c;} u; u.i=66; printf("%c", u.c);`	B (ASCII 66)
6	`printf("%zu", sizeof(int *));`	8 on 64-bit systems

⚠ Common Mistake
(1) Forgetting the **;** after the closing brace of a struct/union; (2) using `.` instead of `->` on a pointer; (3) reading a union member other than the one last written; (4) dereferencing a NULL or uninitialized pointer; (5) `p + 1` on `int *` moves 4 bytes, not 1.

struct	Members	Raw sum	sizeof (typical)	Reason
`{char c; int i;}`	char, int	5	8	3 bytes padding after c
`{int i; char c;}`	int, char	5	8	padding at the end
`{char a; char b; int i;}`	char, char, int	6	8	order members large → small to save space

4.6 More Solved Programs on Pointers, Structures and Unions

Count even and odd numbers using a pointer

```c
int a[6] = {4, 7, 10, 13, 16, 19}, *p = a, even = 0, odd = 0;
while (p < a + 6) {                    // p walks through the array
    if (*p % 2 == 0) even++; else odd++;
    p++;
}
printf("Even=%d Odd=%d", even, odd);
```

OUTPUT

```
Even=3 Odd=3
```

Swapping two variables using pointers

```c
int a = 10, b = 20, *p = &a, *q = &b, t;
t = *p;  *p = *q;  *q = t;              // swap the VALUES through pointers
printf("a=%d b=%d", a, b);
```

OUTPUT

```
a=20 b=10
```

Average marks using an array of structures

```c
struct Student { char name[10]; float marks; };
struct Student s[3] = {{"Ravi", 72.5}, {"Amit", 91.0}, {"Kiran", 65.0}};
float total = 0;  int i;
for (i = 0; i < 3; i++) total += s[i].marks;
printf("Average = %.2f", total / 3);
```

OUTPUT

```
Average = 76.17
```

Structure program: adding complex numbers

```c
struct Complex { float re, im; };
struct Complex a = {3, 4}, b = {1, 2}, c;
c.re = a.re + b.re;   c.im = a.im + b.im;          // (3+4i) + (1+2i)
printf("Sum = %.0f + %.0fi", c.re, c.im);
```

OUTPUT

```
Sum = 4 + 6i
```

Costliest book (array of structures + pointer)

```c
struct Book { char title[15]; float price; };
struct Book b[3] = {{"C Basics", 350}, {"Maths", 520}, {"Physics", 450}};
struct Book *costly = &b[0];  int i;
for (i = 1; i < 3; i++)
    if (b[i].price > costly->price) costly = &b[i];  // best so far
printf("%s costs %.0f", costly->title, costly->price);
```

OUTPUT

```
Maths costs 520
```

Only one union member is valid at a time

```c
union Num { int i; float f; };
union Num n;
n.i = 7;   printf("%d\n", n.i);          // int is valid now
n.f = 2.5; printf("%.1f", n.f);          // float is valid now (int is lost)
```

OUTPUT

```
7
2.5
```

Tricky question	Answer
`int *p, q;`	p is a pointer to int; **q is a plain int** (the * binds to p only)
`int *p = 5;`	Wrong: a pointer must hold an address, not a number
Can a pointer point to a pointer?	Yes: `int **pp = &p;`
Is `p[i]` valid for a pointer?	Yes, it is just `*(p + i)`

★ UNIT SUMMARY — QUICK REVISION

- A **pointer** stores an **address**. `&x` = address of x; `*p` = value at the address in p. Initialize with an address or **NULL**.
- **Pointer arithmetic** is scaled: `p + n` = address + n×sizeof(type). Allowed: ++, --, +n, −n, p2−p1, comparison.
- `a[i]` ≡ `*(a + i)` ≡ `*(p + i)`. Array name is a constant address; a pointer variable can move.
- **Structure**: different types, own memory each, access with **.**; through a pointer use **->**. Can be nested, arrays of struct, and renamed with `typedef`.
- **Union**: shared memory, size = largest member, only one valid member at a time.
- Pointer size = 8 bytes (64-bit) regardless of type; `sizeof(struct)` may include padding.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. What is a pointer? How is it declared?
2. What are the address-of (&) and dereference (*) operators?
3. What is a NULL pointer? Why is it useful?
4. Explain wild and dangling pointers.
5. If `int *p` holds 2000, what is `p + 3`? Why?
6. Write the equivalent pointer expression for `a[i]`.
7. Differentiate between `.` and `->` operators.
8. What is padding in a structure?
9. What is typedef? Give an example.
10. Why does a union need less memory than a structure?

Long Answer / Programs (5–10 marks)

1. Explain pointers with a diagram, covering declaration, initialization and dereferencing.
2. Explain pointer arithmetic with examples. Which operations are not allowed?
3. Write programs using pointers to (a) find the sum and largest of an array, (b) reverse an array, (c) find string length.
4. Define a structure. Write a program to read and print details of n students and find the topper.
5. Explain nested structures and pointer to structure with a program.
6. Explain unions with an example. Compare structure and union with a memory diagram.
7. Explain the relationship between arrays and pointers. How does an array name differ from a pointer?

8. Write a program using an array of structures to store 5 books (title, price) and display the costliest book.

Σ Quick Reference: Syntax Cheat-Sheet

Concept	Syntax	Example
Declare pointer	`type *name;`	`int *p;`
Address of	`&variable`	`p = &x;`
Dereference	`*pointer`	`*p = 25; y = *p;`
Pointer arithmetic	new address = old + n × sizeof(type)	`p + 2` (int: +8 bytes)
Array ↔ pointer	`a[i] = *(a + i)`	`p = a; *(p + 3)`
Define structure	`struct Tag { members; };`	`struct Pt { int x, y; };`
Access member	`var.member` / `ptr->member`	`s.id` / `ps->id`
Define union	`union Tag { members; };`	`union U { int i; float f; };`
Rename a type	`typedef old new;`	`typedef struct {int x,y;} Pt;`